HW1 복습 노트

복습용 참고 자료예요. 좌표는 제가 임의로 잡은 값이라 과제 그림과 다를 수 있고, 같은 원리로 다시 풀어 보는 용도예요.

문제 1. Steiner 정점으로 삼각분할

설정: 바깥 사각형 (0,0)-(12,6), 구멍 두 개 (2,2)-(5,4)와 (7,2)-(10,4). 정점 12개.

Steiner 정점 2개: (6, 0)과 (6, 6). 바깥 위쪽 변과 아래쪽 변 위, 두 구멍 사이 막대의 가운데예요. 도형의 경계 변 위라서 허용돼요.

왜 여기인가

- Steiner 없이 12개 정점만으로 제약 Delaunay 삼각분할을 하면 삼각형 14개이고 **가장 작은 각이 5.9도**예요. 가장 안 좋은 삼각형은 (0,6), (5,4), (7,4)예요. 바깥 왼쪽 위 코너에서 구멍 사이 막대 위쪽의 짧은 변(길이 2)까지 길게 뻗은 가늘고 긴 삼각형이에요.
- 막대 바로 위의 바깥 변에 점 (6,6)을 찍고(아래쪽 (6,0)도 같은 이유), 그 점에서 막대로 삼각형을 이으면 이런 삼각형이 사라져서 **가장 작은 각이 18.4도**로 좋아져요.
- 구멍 가운데나 구멍 사이 막대 안쪽에 점을 찍어 보면(예: (6,1)과 (6,5)) 오히려 9.5도로 더 나빠졌어요. 점이 있어야 할 곳은 긴 변 위예요.
- 18.4도가 나오는 배치가 여러 개 있어서, 대칭인 (6,0), (6,6)을 골랐어요.

삼각형 수 검산: 삼각형 수 = B + 2h − 2 + 2I (경계 정점 B, 구멍 수 h, 내부 정점 I)

- 경계 정점은 12 + 2 = 14, 구멍 2개, 내부 정점 0 → 14 + 4 − 2 = **16개** (코드 출력과 일치)

삼각형 목록 (정점 번호, 모두 반시계)

```
{0,4,7} {0,12,4} {2,10,9} {4,12,5} {5,12,8} {7,3,0} {8,6,5} {9,1,2}
{9,12,1} {10,2,13} {11,6,8} {12,9,8} {13,3,7} {13,6,11} {13,7,6} {13,11,10}
```

번호: 0~3 바깥(왼아래, 오른아래, 오른위, 왼위), 4~7 왼쪽 구멍, 8~11 오른쪽 구멍, 12 = (6,0), 13 = (6,6)

확인한 것

- 16개 모두 CCW. 그래서 `glPolygonMode(GL_FRONT, GL_FILL)`로 전부 채워져요.
- 삼각형 넓이 합 60 = 도형 넓이 72 − 6 − 6 = 60. 겹침이나 빈틈이 없어요.
- 삼각형끼리 교차하지 않고 구멍 안에 삼각형이 없어요.

문제 2. 카메라 축과 뷰 행렬

설정: E = (4, 3, 0), A = (0, 0, 0), Up = (0, 1, 0)

손 계산

```
los   = A − E = (−4, −3, 0)
z_cam = (E − A)/|E − A| = (4, 3, 0)/5 = (0.8, 0.6, 0)
x_cam = Up × z_cam = (0,1,0) × (0.8,0.6,0)
        x: 1 · 0 − 0 · 0.6   = 0
        y: 0 · 0.8 − 0 · 0   = 0
        z: 0 · 0.6 − 1 · 0.8 = −0.8
      = (0, 0, −0.8) → 정규화 (0, 0, −1)
y_cam = z_cam × x_cam = (0.8,0.6,0) × (0,0,−1)
        x: 0.6 · (−1) − 0 · 0 = −0.6
        y: 0 · 0 − 0.8 · (−1) = 0.8
        z: 0.8 · 0 − 0.6 · 0  = 0
      = (−0.6, 0.8, 0)
```

V = R × T

```
T = translate(−4, −3, 0)

R =  0    0   −1   0
    −0.6  0.8  0   0
     0.8  0.6  0   0
     0    0    0   1

V 마지막 열 = −(x_cam · E, y_cam · E, z_cam · E)
  x_cam · E = 0
  y_cam · E = −0.6 · 4 + 0.8 · 3 = 0
  z_cam · E = 0.8 · 4 + 0.6 · 3 = 5

V =  0    0   −1    0
    −0.6  0.8  0    0
     0.8  0.6  0   −5
     0    0    0    1
```

검산: V × E = (0, 0, 0), V × A = (0, 0, −5). 5 = |E − A|.

코드와 비교: `glm::lookAt(E, A, Up)` 결과와 같아요. (최대 차이 0) `glm::mat4`는 열 우선이라 출력할 때 `m[열][행]`으로 읽어야 손 계산과 같은 모양이 나와요.

문제 3. triangle strip 한 번으로 사각 고리

설정: 바깥 정사각형 (0,0)-(10,10), 안쪽 정사각형 (3,3)-(7,7)

삼각형 수: 고리를 코너 8개만으로 삼각분할하면 삼각형이 B + 2h − 2 = 8 + 2 − 2 = **8개**예요. strip은 정점 k개가 삼각형 k − 2개를 만들어서 k = 10, **최소 정점 수는 10개**예요.

정점 순서: 바깥과 안쪽을 번갈아 한 바퀴 돌고, 마지막에 처음 두 점을 한 번 더 써서 고리를 닫아요.

```
(0,0) (3,3) (10,0) (7,3) (10,10) (7,7) (0,10) (3,7) (0,0) (3,3)
```

코너 8개 + 반복 2개 = 10개

orientation: strip은 홀수 번째 삼각형의 앞 두 점을 바꿔서 취급해요. 8개 삼각형의 외적 z 부호를 계산하면 ** 전부 같은 방향(CW)**이에요. 그래서 **같은 orientation으로 만들 수 있어요.** (정점을 반대 방향으로 돌면 전부 CCW가 돼요.)

확인한 것: 삼각형 넓이 합 84 = 100 − 16 = 고리 넓이. 겹침과 빈틈이 없어요.

문제 4. gluLookAt에 해당하는 modeling 변환

(a) gluLookAt(0, 0, 0, −15, 0, 15, 0, 1, 0)

```
eye = (0,0,0), center = (−15,0,15), up = (0,1,0)

los   = center − eye = (−15, 0, 15)
z_cam = (eye − center)/|…| = (15, 0, −15)/(15√2) = (1/√2, 0, −1/√2)
x_cam = up × z_cam = (0,1,0) × (a, 0, −a)       (a = 1/√2)
        x: 1·(−a) − 0·0 = −a
        y: 0·a − 0·(−a) = 0
        z: 0·0 − 1·a   = −a
      = (−a, 0, −a)       (크기 1)
y_cam = z_cam × x_cam = (a, 0, −a) × (−a, 0, −a)
        x: 0·(−a) − (−a)·0 = 0
        y: (−a)(−a) − a·(−a) = a² + a² = 1
        z: a·0 − 0·(−a) = 0
      = (0, 1, 0)
```

eye가 원점이라 T = 단위 행렬이고 **V = R**이에요.

```
R =  −a  0  −a  0
      0  1   0  0
      a  0  −a  0
      0  0   0  1
```

y축 회전 행렬 `[cos θ 0 sin θ ; 0 1 0; −sin θ 0 cos θ ]`와 비교하면 cosθ = −1/√2, sinθ = −1/√2라서 θ = **225도** (또는 −135도)예요.

modeling 변환: `glRotatef(225, 0, 1, 0)` (y축 기준 225도 회전)

검산: V × center = R × (−15, 0, 15) = (0, 0, −30a) = (0, 0, −21.2). 거리 |center| = 15√2 = 21.2, 방향은 −z ✓

(b) gluLookAt(0, 0, 15, 0, 0, 0, 1, 0, 0)

up이 (1, 0, 0)이라는 점이 (a)와 달라요.

```
eye = (0,0,15), center = (0,0,0), up = (1,0,0)

los   = (0, 0, −15)
z_cam = (0, 0, 15)/15 = (0, 0, 1)
x_cam = up × z_cam = (1,0,0) × (0,0,1)
        x: 0 · 1 − 0 · 0 = 0
        y: 0 · 0 − 1 · 1 = −1
        z: 1 · 0 − 0 · 0 = 0
      = (0, −1, 0)
y_cam = z_cam × x_cam = (0,0,1) × (0,−1,0)
        x: 0 · 0 − 1 · (−1) = 1
        y: 1 · 0 − 0 · 0    = 0
        z: 0 · (−1) − 0 · 0 = 0
      = (1, 0, 0)
```

카메라 축이 세계 축과 달라요. x_cam이 −y, y_cam이 +x 방향이에요. 즉 카메라가 시선축(z)을 중심으로 **옆으로 돌아간** 거예요.

```
R =  0 −1  0  0
     1  0  0  0
     0  0  1  0
     0  0  0  1
T = translate(0, 0, −15)
V = R × T
```

R을 z축 회전 행렬 `[cos θ −sin θ ; sin θ cos θ]` 와 비교하면 cosθ = 0, sinθ = 1이라서 θ = **90도**예요.

modeling 변환 (코드 순서는 적용 순서의 반대)

```
glRotatef(90, 0, 0, 1);
glTranslatef(0, 0, -15);
```

검산: V × eye = R × (0, 0, 0) = (0, 0, 0) ✓, V × center = R × (0, 0, −15) = (0, 0, −15) ✓ (−z축 위, 거리 15)

정리: 이 과제에서 쓴 개념

문제	개념	복습 위치
1	triangulation, long thin triangle, 삼각형 수 식, orientation	4단계(삼각분할)
2	카메라 좌표계 (z → x → y 외적 순서), V = R·T, V×eye = 0	5단계
3	triangle strip (k 정점 → k−2 삼각형), strip의 orientation 규칙	4단계(strip), 외적(4단계)
4	뷰 행렬을 이동과 회전 변환으로 해석, 코드는 적용 순서의 반대	5단계, 7단계(복합 변환)